PROJECT DOCUMENTATION

End-to-End CI/CD Pipeline with GitOps on AWS EKS

Mini Accounting App | Jenkins | SonarQube | Trivy
Docker Hub | Argo CD | Kubernetes (Amazon EKS)

PREPARED BY	**Waqas**
DATE	**4 November 2026**
CLOUD PLATFORM	**Amazon Web Services (Mumbai & Hyderabad regions)**
DOCUMENT TYPE	**Technical walkthrough with evidence screenshots**

Table of Contents

Table of Contents 2

1. Project Overview 3

2. Architecture and Workflow 4

3. Cloud Infrastructure on AWS 4

4. Source Code Repository 7

5. Continuous Integration Pipeline 7

6. Container Registry (Docker Hub) 11

7. Continuous Delivery (GitOps) Pipeline 12

8. GitOps Repository and Argo CD 14

9. Deployed Application 16

10. End-to-End Flow Summary 16

11. Observations and Recommendations 17

12. Conclusion 17

1. Project Overview

This project implements a complete, automated software delivery pipeline for a small Java application called the **Mini Accounting App**, which tracks income and expenses and exposes simple REST endpoints. Every code change pushed to GitHub is automatically built, tested, analysed for quality and security, packaged as a Docker image, published to Docker Hub and finally deployed to a Kubernetes cluster on **Amazon EKS** using the **GitOps** approach with Argo CD.

Objectives

- Automate build, test and packaging so that no manual steps are needed between commit and deployment.
- Enforce code quality with a SonarQube Quality Gate before an image is ever produced.
- Detect known vulnerabilities in the container image using Trivy.
- Use Git as the single source of truth for what runs in Kubernetes (GitOps) and let Argo CD keep the cluster in sync.
- Run the whole toolchain on AWS and expose the final application publicly through an AWS Load Balancer.

Technology Stack

Layer	Tool	Purpose
Source control	GitHub	Hosts application code (Accounting-app) and Kubernetes manifests (register-app)
CI / CD engine	Jenkins (controller + agent)	Orchestrates the CI and CD pipelines
Build	Maven, Java	Compiles, tests and packages the application
Code quality	SonarQube Community	Static analysis and Quality Gate
Container	Docker, Docker Hub	Builds and stores the application image
Security	Trivy	Scans the image for vulnerabilities
GitOps	Argo CD	Continuously syncs the cluster with the manifests in Git
Orchestration	Amazon EKS (Kubernetes)	Runs 2 replicas of the app
Infrastructure	AWS EC2, CloudFormation (eksctl), ELB	Hosts tools, creates the cluster, exposes the app

2. Architecture and Workflow

The design is split into two halves. **Continuous Integration (CI)** verifies and packages the code. **Continuous Deployment (CD)** uses the GitOps pattern: instead of Jenkins pushing directly to the cluster, it only updates the image tag in a Git repository, and Argo CD pulls that change into Kubernetes.

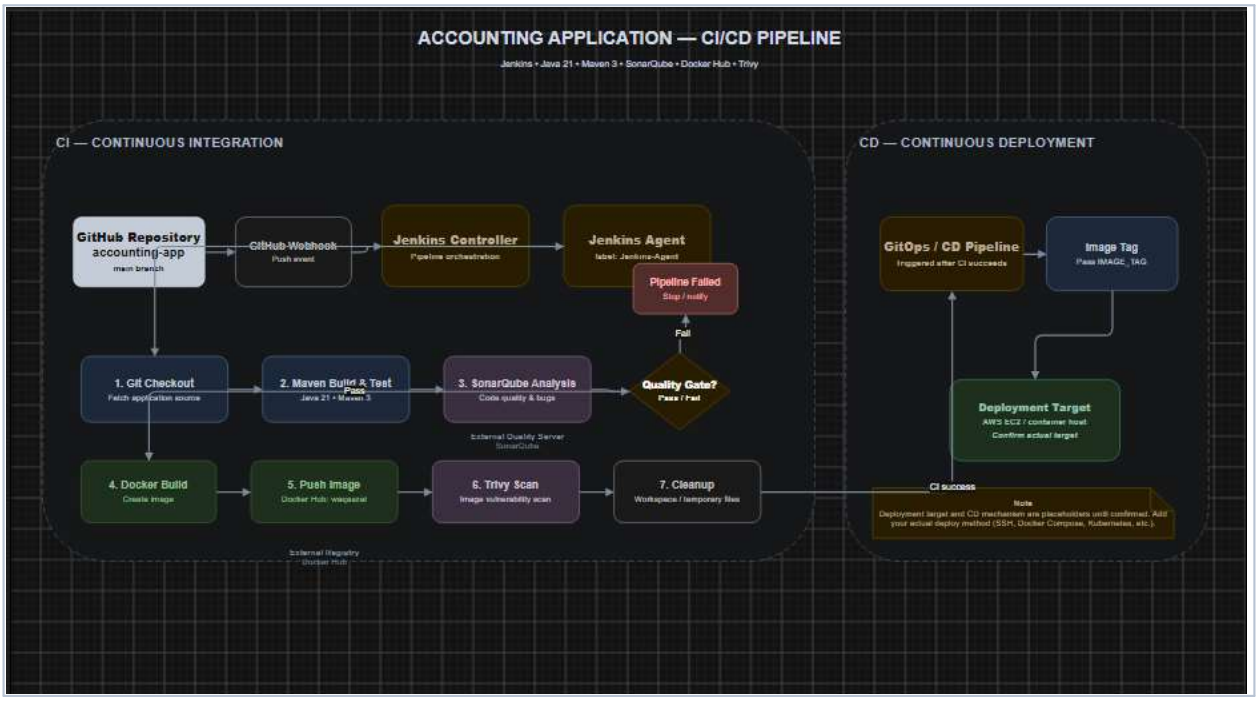


Figure 1: Architecture of the CI/CD pipeline (CI on the left, CD on the right).

How the flow works

- **GitHub push** on the main branch fires a webhook to the Jenkins controller.
- The **Jenkins controller** schedules the job on the **Jenkins agent**, where the work actually runs.
- The agent runs the CI stages: checkout, Maven build and test, SonarQube analysis, Quality Gate, Docker build and push, Trivy scan and cleanup.
- If the Quality Gate fails, the pipeline stops and notifies; if it passes, the pipeline continues.
- On success, CI triggers the **GitOps CD pipeline** and passes the image tag as a parameter.
- The CD pipeline updates the deployment manifest in the GitOps repository; **Argo CD** detects the commit and deploys the new version to EKS.

3. Cloud Infrastructure on AWS

3.1 Tooling servers (Asia Pacific - Hyderabad)

The DevOps toolchain runs on four EC2 instances of type **m7i-flex.large** in the **ap-south-2** region. All four pass their 3/3 status checks and no alarms are raised.

Instance name	Role
Jenkins-Master	Jenkins controller: hosts the UI, stores job configuration and schedules builds
Jenkins-Agent	Build executor (label Jenkins-Agent): runs Maven, Docker and Trivy

Instance name	Role
SonarQube	Runs the SonarQube server used for code analysis and the Quality Gate
EKS-Bootstrap	Management host with the AWS CLI, eksctl and kubectl used to create and administer the cluster

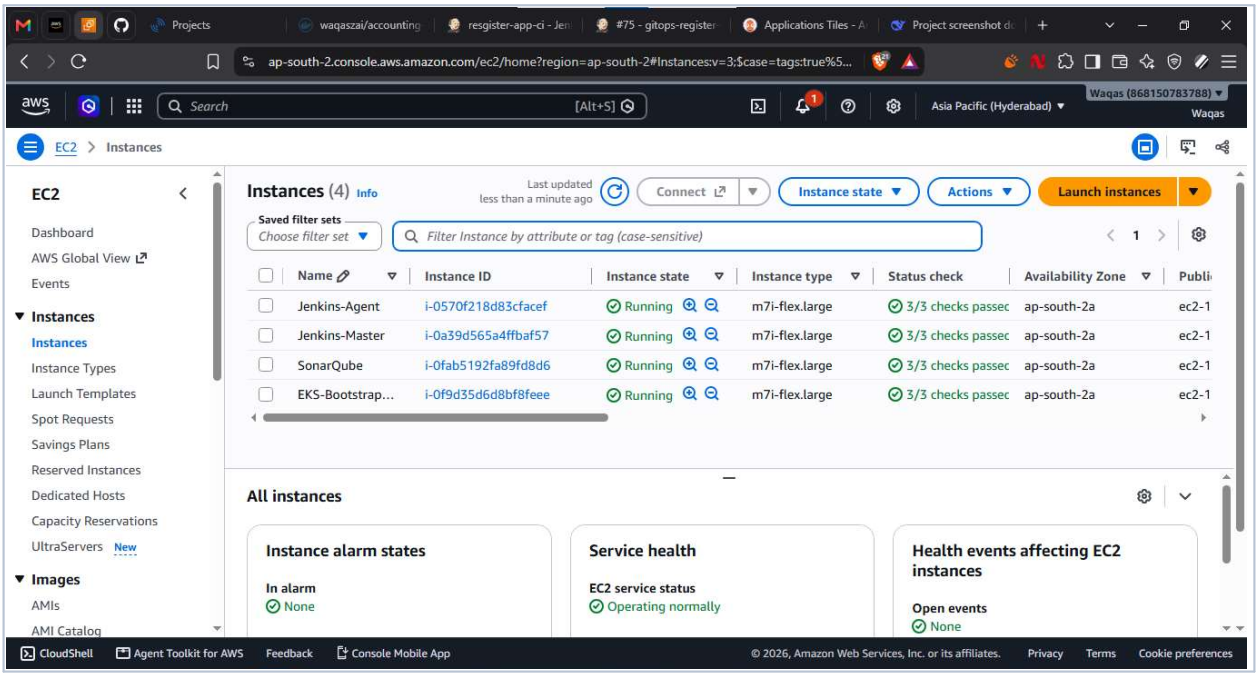


Figure 2: EC2 instances hosting Jenkins, SonarQube and the EKS bootstrap host (ap-south-2).

3.2 Amazon EKS cluster (Asia Pacific - Mumbai)

The Kubernetes cluster is created with **eksctl**, which uses AWS CloudFormation behind the scenes. Two stacks were created: one for the EKS control plane and one for the managed node group. Both show **CREATE_COMPLETE**. The control plane stack was created at 16:25 and the node group at 16:35 (UTC+05:30) on 4 November 2026.

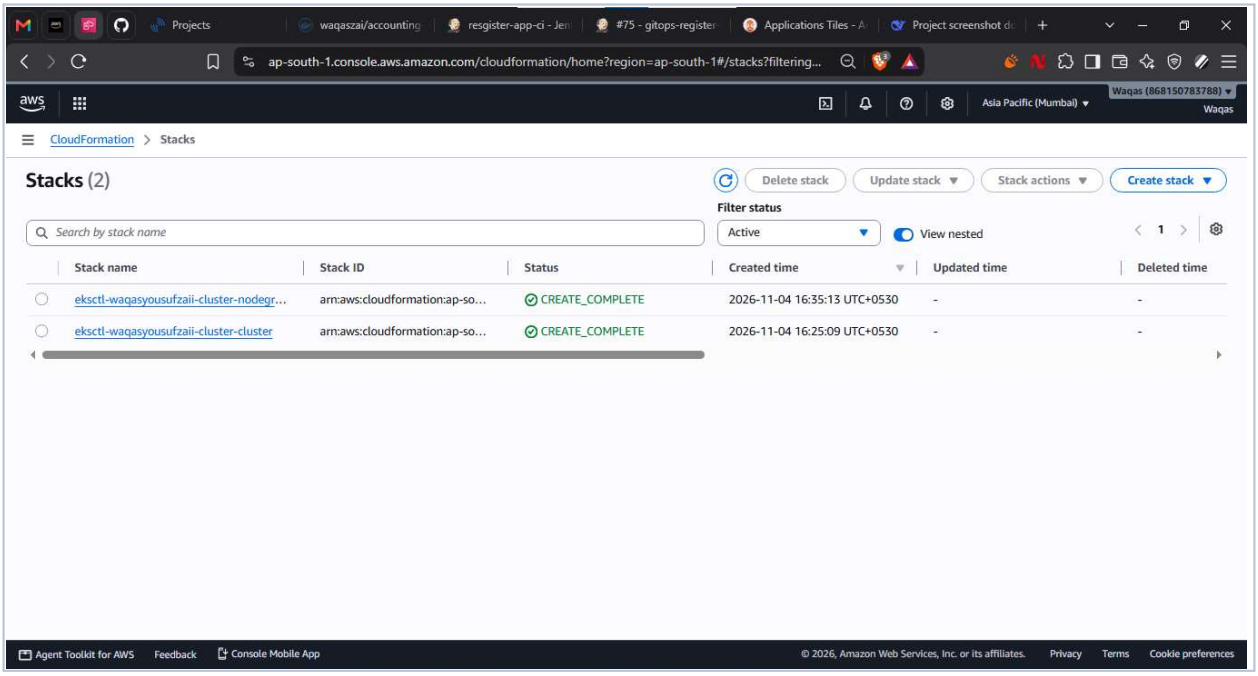


Figure 3: CloudFormation stacks created by eksctl for the cluster and node group.

The node group provides three **t3.small** worker nodes (ap-south-1) that run the application pods, all in the Running state with 3/3 status checks passed.

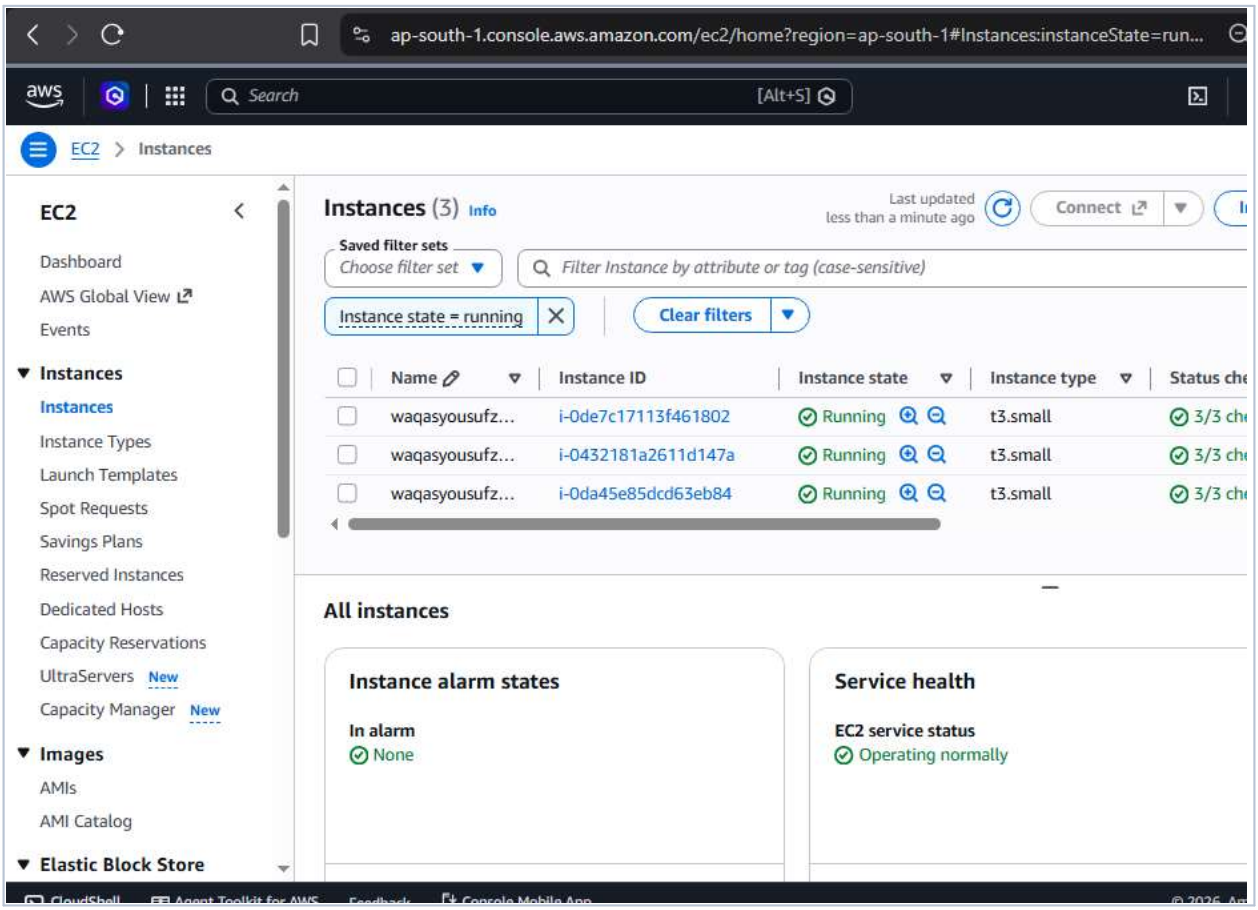


Figure 4: EKS worker nodes (three t3.small EC2 instances) running in ap-south-1.

4. Source Code Repository

The application lives in the public GitHub repository **Accounting-app**. It is a small Java and Maven application with no external libraries. Besides the source code under `src`, the repository contains everything required to build and ship it:

File	Purpose
Jenkinsfile	Declarative pipeline that defines all CI stages
Dockerfile	Instructions to package the compiled application into a container image
docker-compose.yml	Optional local run configuration
pom.xml	Maven project definition: dependencies, build and test configuration
.dockerignore / .gitignore	Exclude unneeded files from the image and from Git
README.md	Project description

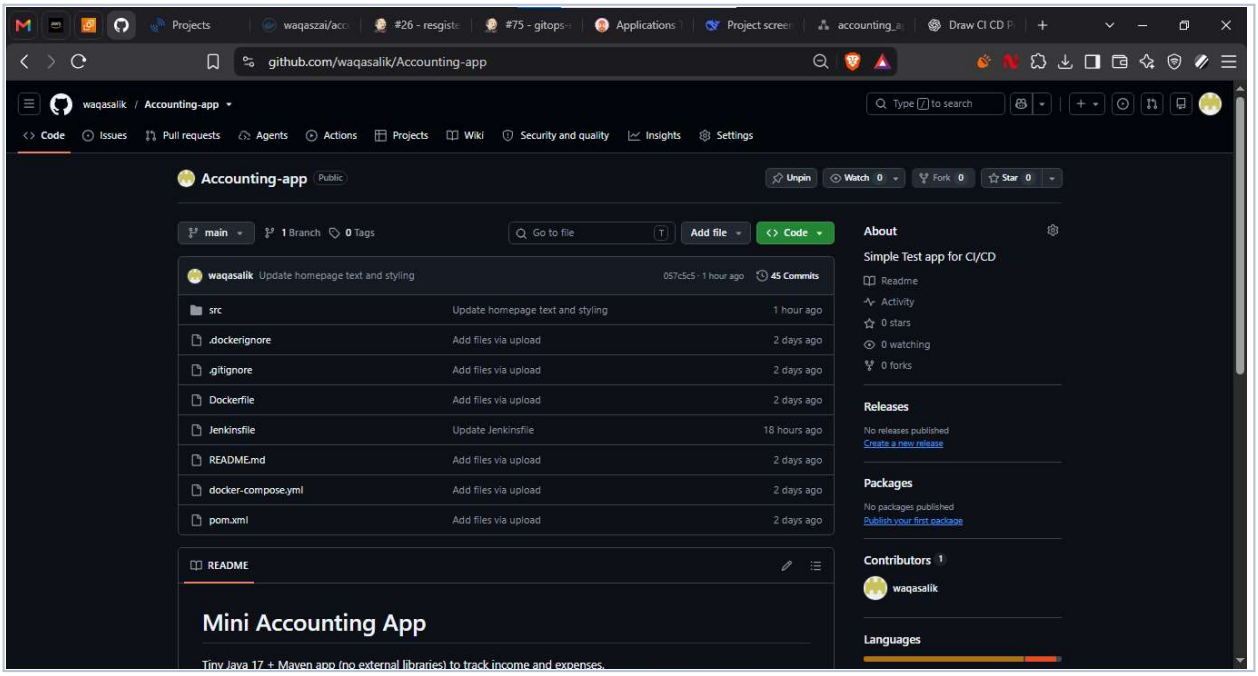


Figure 5: GitHub repository Accounting-app (main branch, 45 commits).

5. Continuous Integration Pipeline

5.1 Trigger and job overview

The CI job (**resgister-app-ci** in Jenkins) is started automatically by a GitHub push through a webhook. The job page shows the SonarQube Quality Gate result directly in Jenkins, together with permalinks to the last, last stable, last successful and last failed builds. Build **#26** is the last successful build; build #25 was an earlier unsuccessful attempt that was fixed before #26.

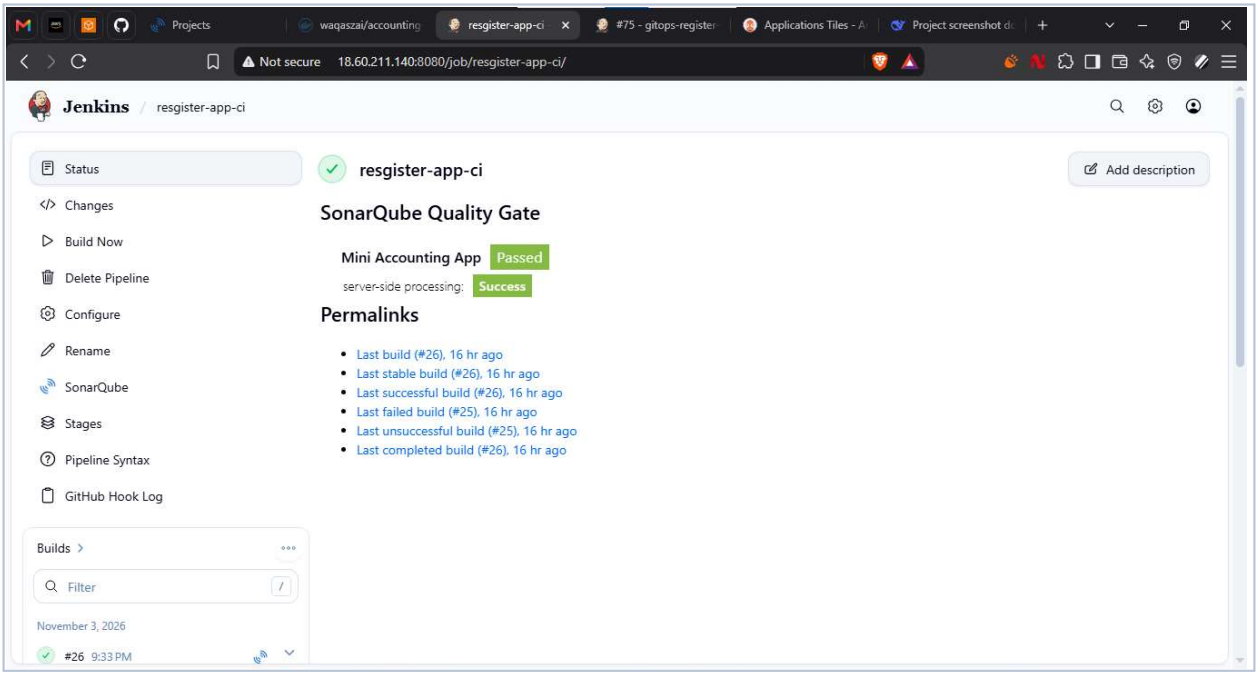


Figure 6: Jenkins CI job page showing the SonarQube Quality Gate: Passed.

5.2 Pipeline stages

Build #26 was started by a GitHub push and completed in about 2 minutes 18 seconds. Every stage is green.

#	Stage	What it does
1	Checkout SCM	Jenkins pulls the pipeline definition (Jenkinsfile) from GitHub
2	Tool Install	Provisions the Java 21 and Maven 3 tools configured in Jenkins
3	Cleanup Workspace	Removes leftovers from earlier runs to guarantee a clean build
4	Checkout from SCM	Fetches the application source code
5	Build Application	Compiles and packages the app with Maven
6	Test Application	Runs the unit tests
7	SonarQube Analysis	Sends the code to SonarQube for static analysis
8	Quality Gate	Waits for SonarQube's verdict; the build stops if the gate fails
9	Build & Push Docker Image	Builds the image and pushes it to Docker Hub with a version tag (for example 1.0.0-26) and latest
10	Trivy Scan	Scans the image for known vulnerabilities
11	Cleanup Artifacts	Deletes temporary files and local images from the agent
12	Trigger GitOps Pipeline	Calls the CD job through the Jenkins API (curl) and passes IMAGE_TAG=1.0.0-26

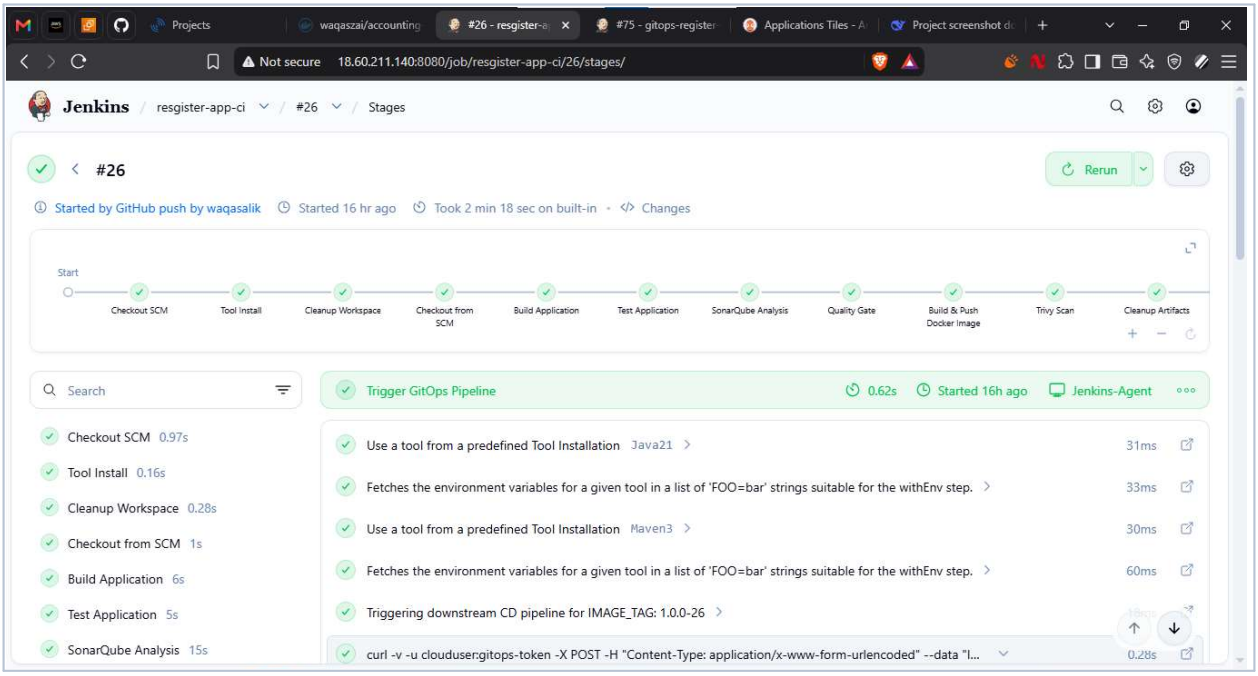


Figure 7: Stage view of CI build #26: all stages passed, including the downstream trigger.

5.3 Code quality with SonarQube

SonarQube analyses the source code for bugs, vulnerabilities and maintainability problems. The project **Mini Accounting App** (232 lines of Java and XML) **passed** its Quality Gate with ratings of **A for Security, Reliability and Maintainability**, zero new issues and 0.0% duplication.

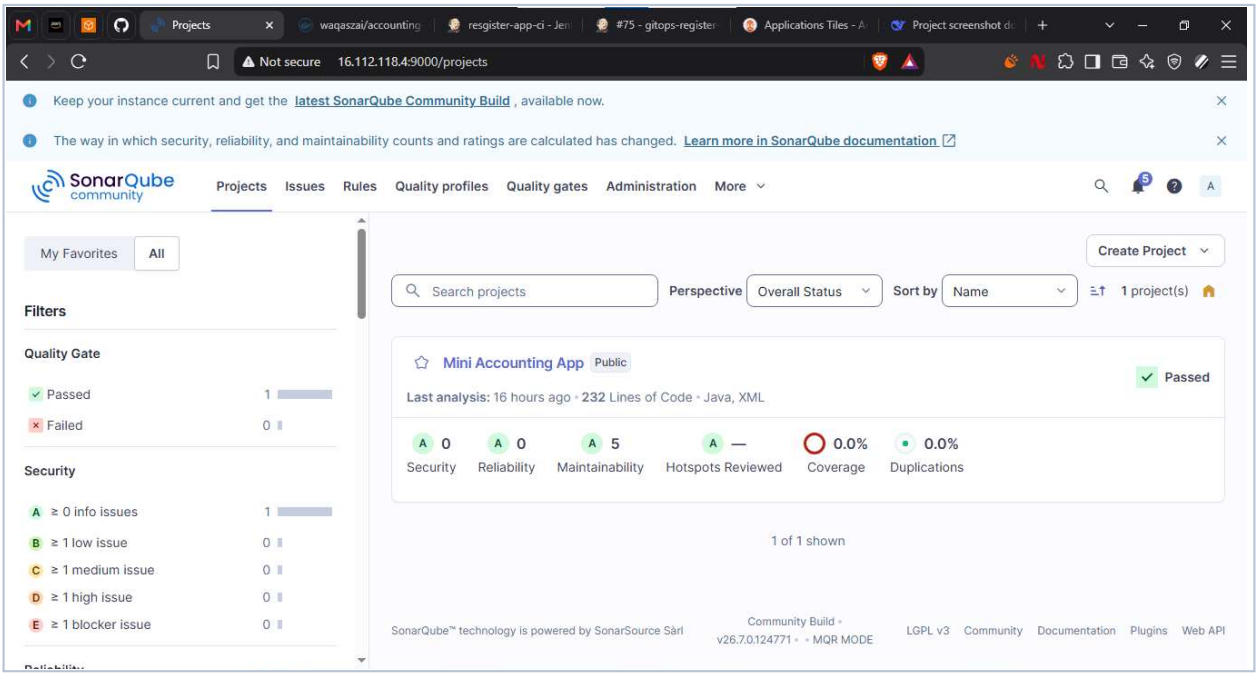


Figure 8: SonarQube projects page: Quality Gate Passed, Security A, Reliability A, Maintainability A.

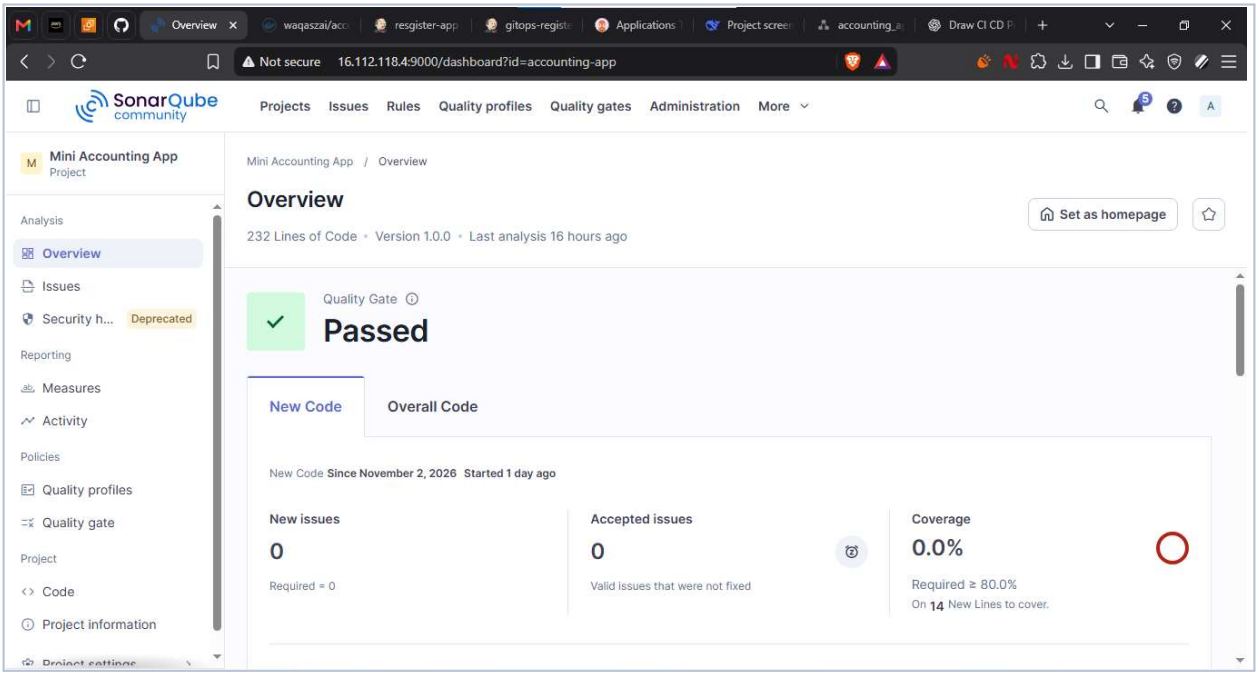


Figure 9: SonarQube project overview: Quality Gate Passed with 0 new issues.

Note: code coverage is 0.0% because the project does not yet contain coverage-measured unit tests. Adding tests with a coverage plugin such as JaCoCo is the most valuable next improvement (see Section 11).

5.4 Container security with Trivy

After the image is built, **Trivy** scans every layer for known CVEs and prints a table in the Jenkins console, listing the library, vulnerability ID, severity, installed version and a short description with a link to the advisory. The findings in this build are in operating-system packages of the base image (for example systemd, glibc locales, shadow-utils, p11-kit and rust-coreutils) and are rated **LOW** or **MEDIUM**; none of them were reported as HIGH or CRITICAL in the portion of the output captured.

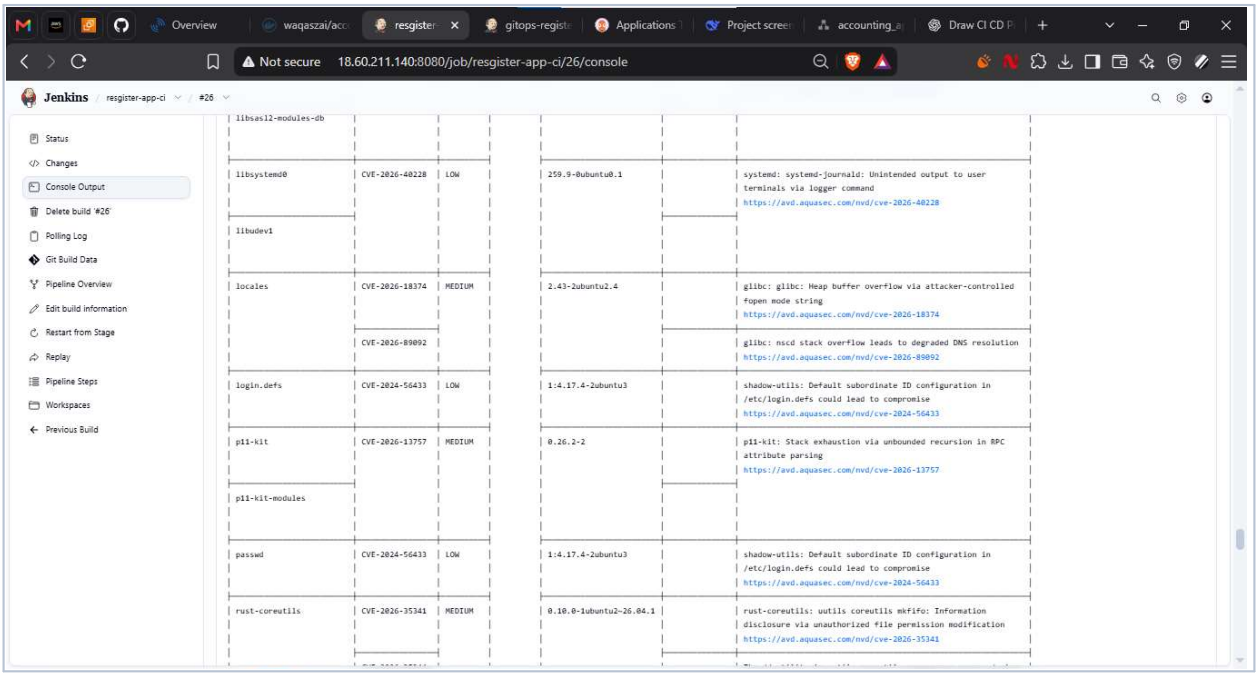


Figure 10: Trivy vulnerability report in the Jenkins console output of CI build #26.

5.5 CI result

The end of the console log shows the output of the application's HTTP response (served by Jetty) followed by the closing of all pipeline blocks and the line **Finished: SUCCESS**, confirming the CI run completed without errors.

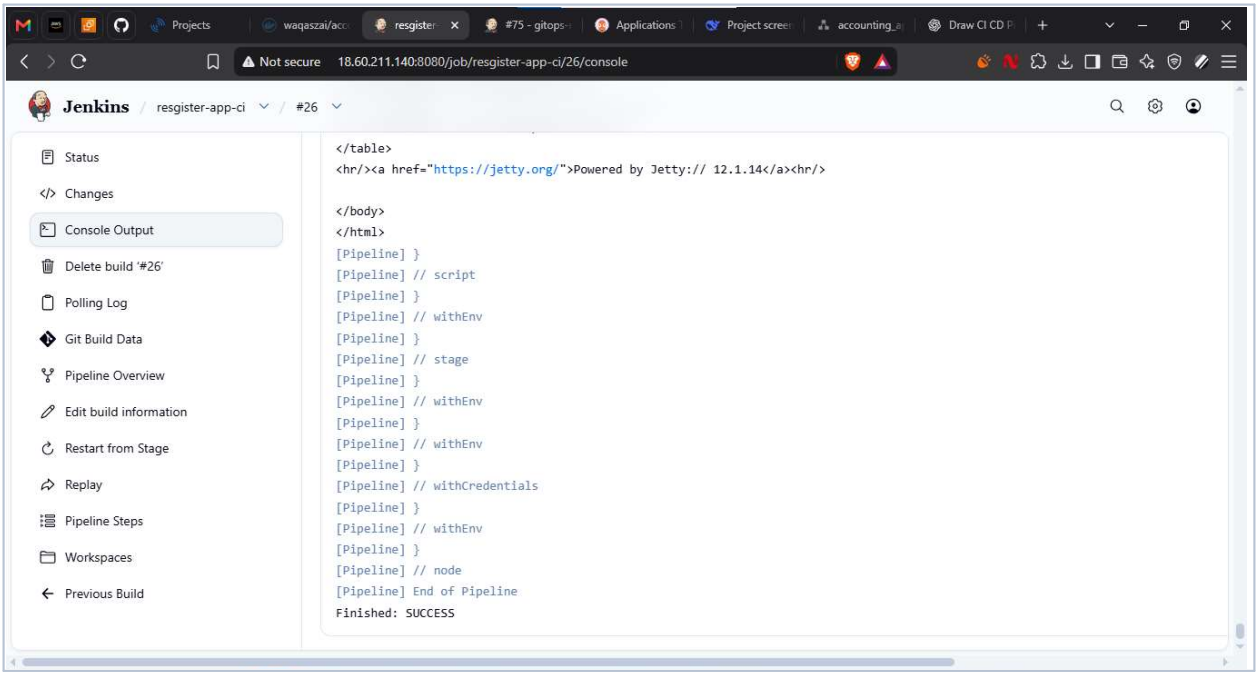


Figure 11: Console output of build #26 ending with Finished: SUCCESS.

6. Container Registry (Docker Hub)

Images are published to the public Docker Hub repository **waqaszai/accounting-app**. Each CI run pushes an image tagged with the application version and the Jenkins build number (`1.0.0-24`, `1.0.0-25`, `1.0.0-26`) and also updates the `latest` tag. Unique tags make every deployment traceable to a specific build and allow quick rollbacks.

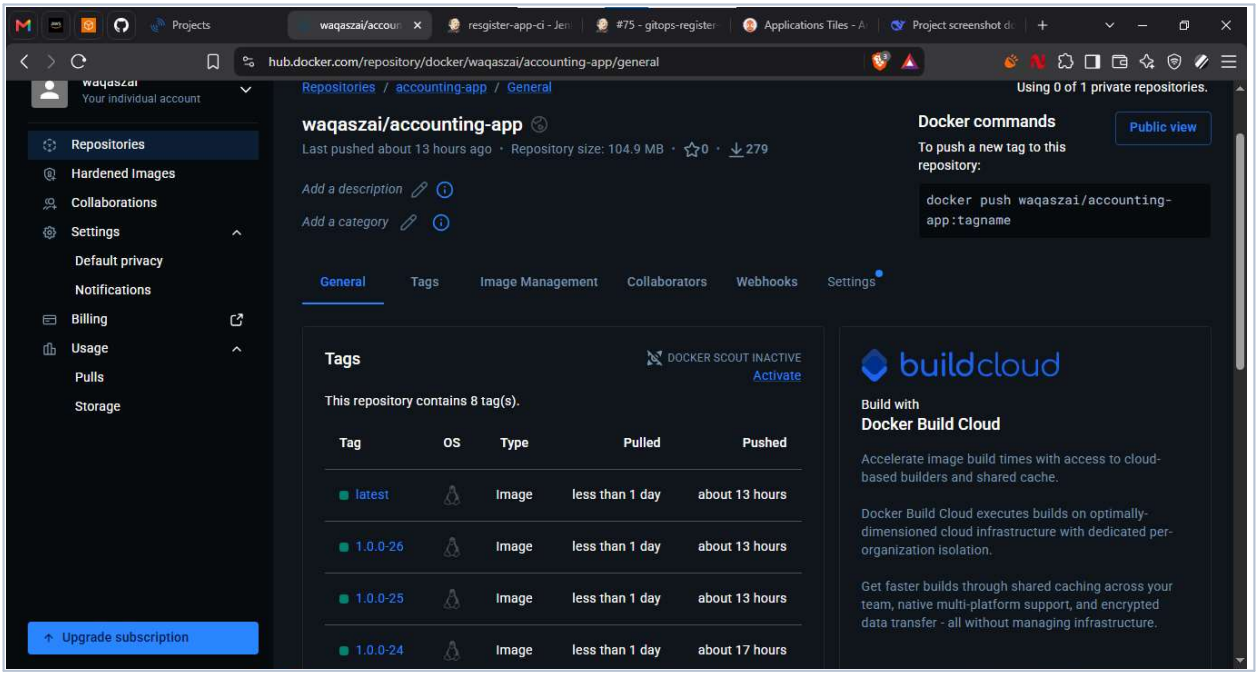


Figure 12: Docker Hub repository after the CI pipeline: version tags 1.0.0-24, -25, -26 and latest (8 tags).

7. Continuous Delivery (GitOps) Pipeline

The second Jenkins job, **gitops-register-app-cd**, is triggered by the CI pipeline (or manually by a user). Its history shows several runs: #73 and #75 succeeded, #74 failed and was corrected, and #72 is marked as the last stable build. This is typical of pipeline development, where failed runs are fixed and rerun.

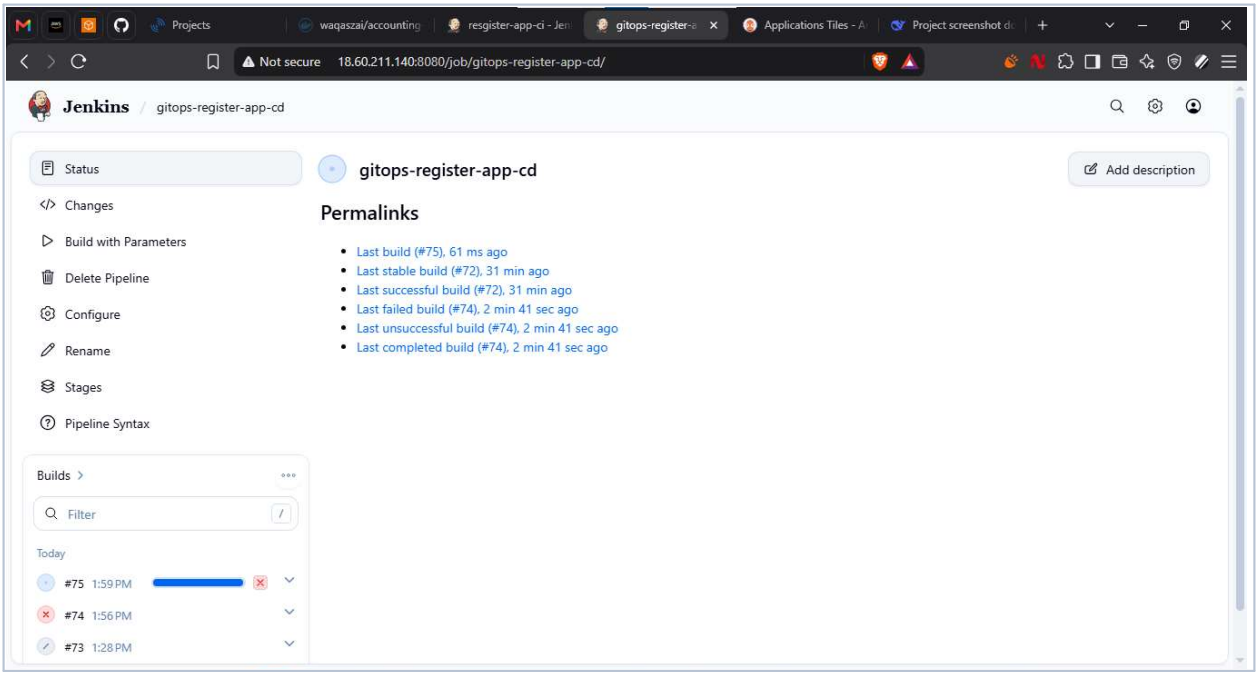


Figure 13: Jenkins CD job gitops-register-app-cd with its build history and permalinks.

Build **#75** completed in 23 seconds. Its stages are: Checkout SCM, Cleanup Workspace, Checkout App Code, Build Docker Image, Trivy Scan, Push Docker Image, **Update GitOps Manifest**, Push the changed deployment file to Git, and Post Actions. The key idea is the manifest update: the pipeline edits

the image tag in `deployment.yaml` and commits the change, which is what ultimately triggers the deployment.

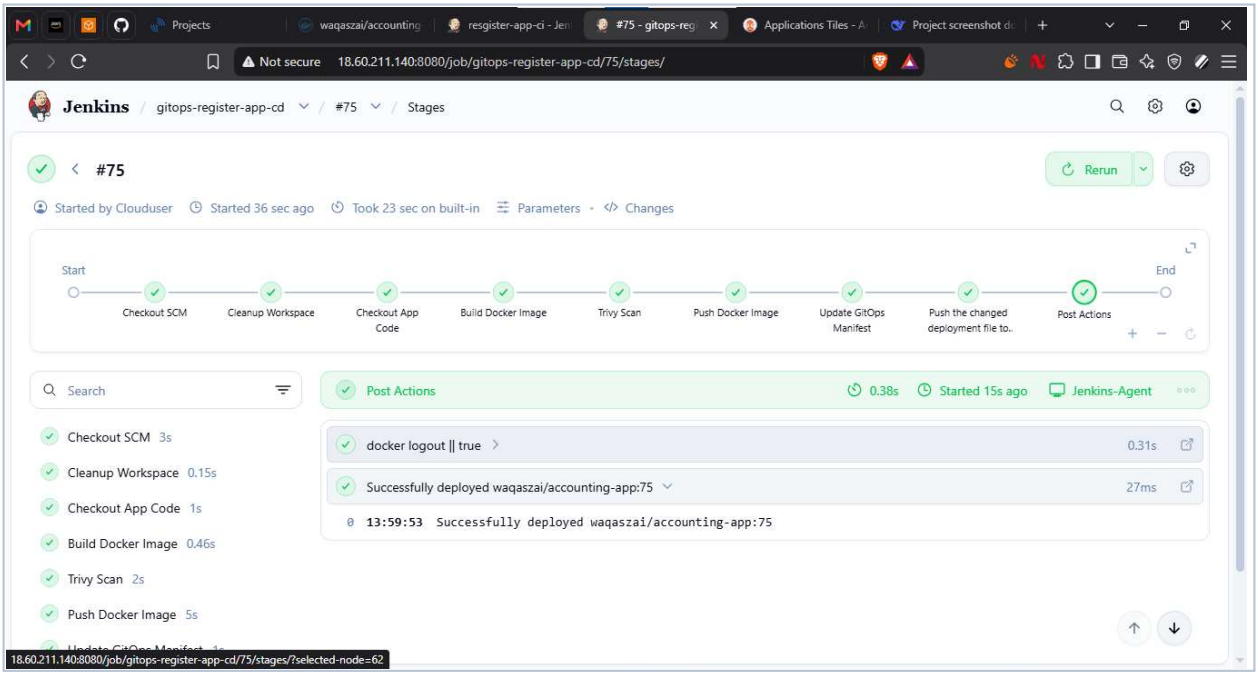


Figure 14: Stage view of CD build #75: all stages successful.

The console output proves the GitOps step. Jenkins commits *Update image tag to 75 [ci skip]* to the GitOps repository and pushes it to the `main` branch using stored credentials (Jenkins masks the secret values in the log). The `[ci skip]` marker prevents the commit from starting another pipeline run and creating a loop. The post action logs out of Docker Hub and prints *Successfully deployed waqaszai/accounting-app:75*.

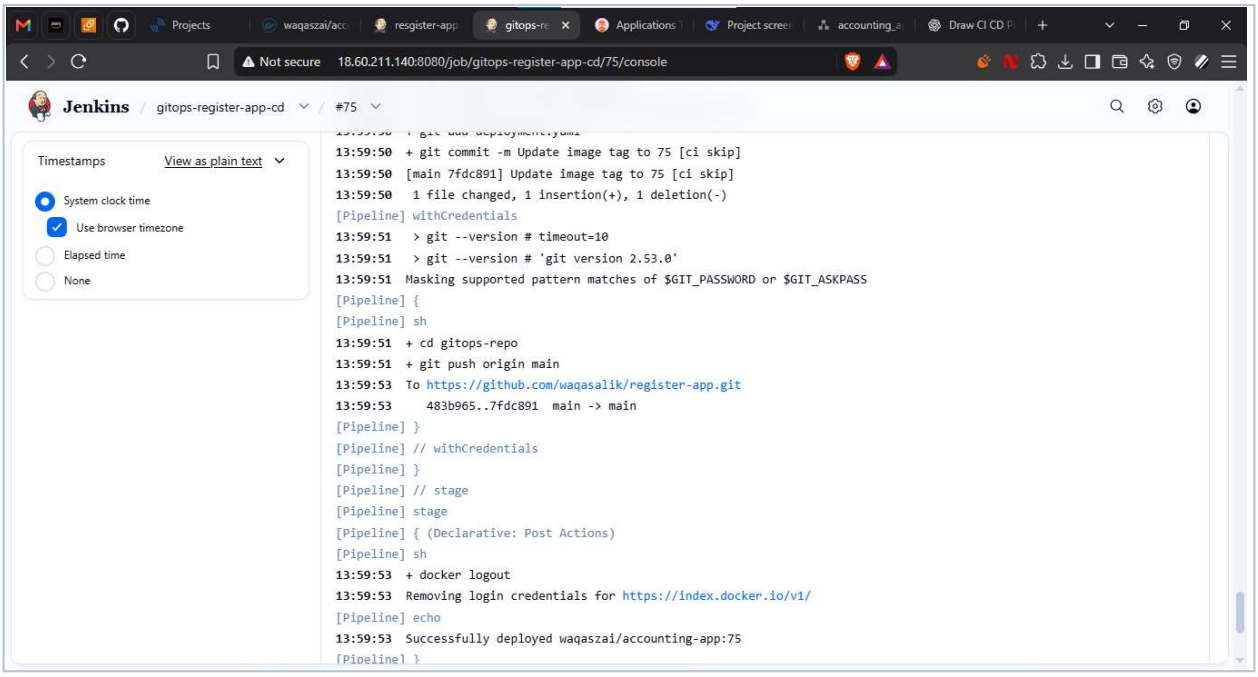


Figure 15: Console of CD build #75: manifest commit, push to GitHub and Docker logout.

The new image tag appears on Docker Hub within minutes. The repository now holds 12 tags, including the CD build tags **75**, **63**, **51** and **14**.

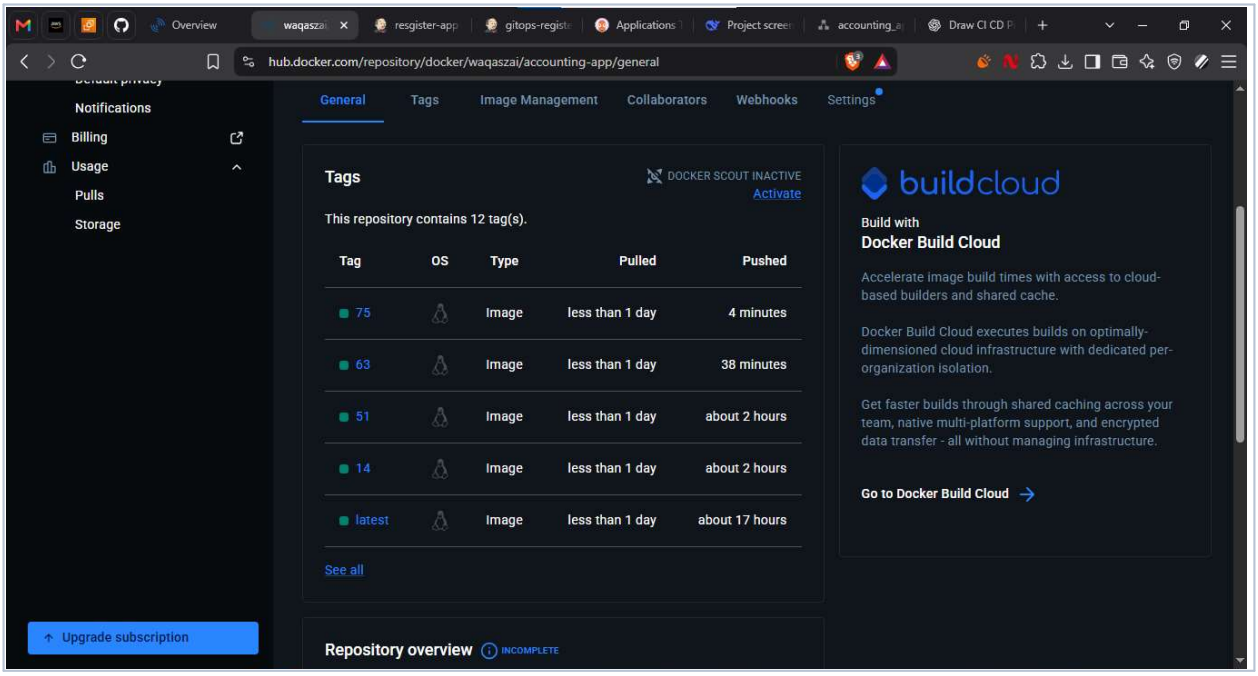


Figure 16: Docker Hub after the CD pipeline: tag 75 pushed minutes earlier (12 tags in total).

8. GitOps Repository and Argo CD

8.1 GitOps repository (register-app)

The repository **register-app** contains only Kubernetes manifests and acts as the desired state of the cluster:

File	Purpose
deployment.yaml	Runs 2 replicas of the application image from Docker Hub; Jenkins updates the image tag here
service.yaml	Creates a Service of type LoadBalancer so AWS provisions a public Elastic Load Balancer in front of the pods
Jenkinsfile	Pipeline definition of the CD job
README.md	Documentation of the GitOps layout

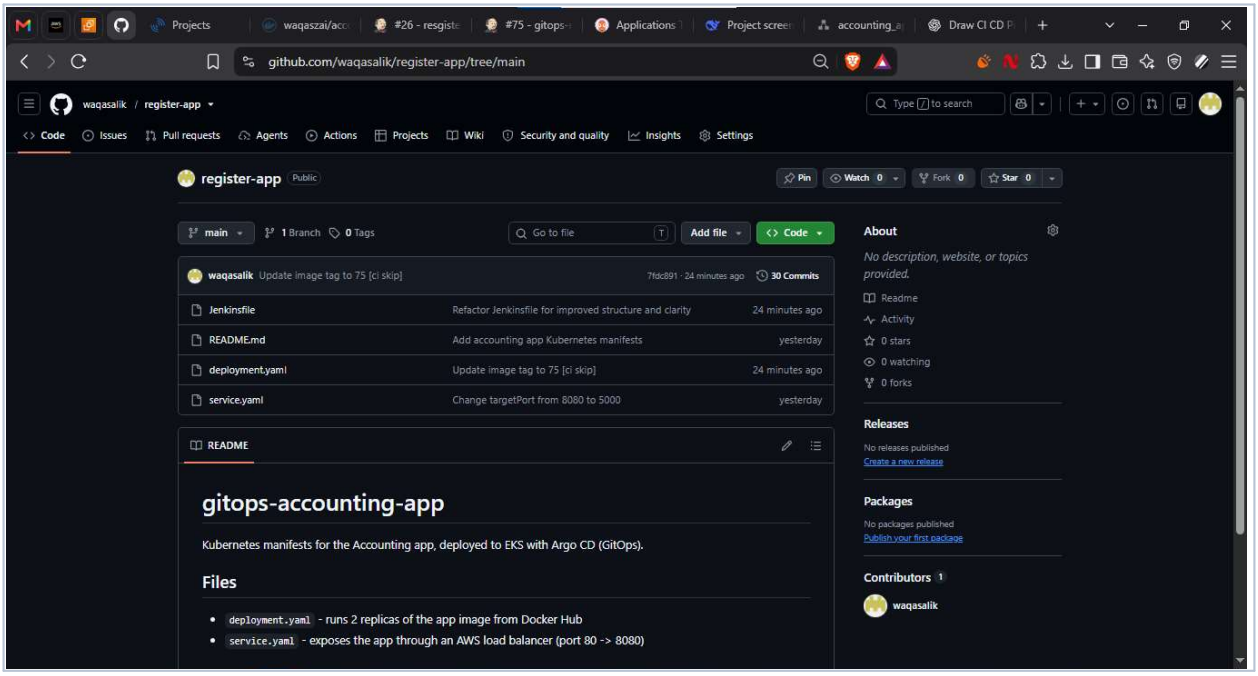


Figure 17: GitOps repository: latest commit 'Update image tag to 75 [ci skip]' made by Jenkins.

8.2 Argo CD

Argo CD (v3.6.0) runs inside the cluster and is reachable through its own load balancer. The application **register-app** is configured with the GitOps repository as source, target revision **HEAD**, path `./`, destination cluster *waqasyousufzaii-cluster* and namespace *default*. It reports **Healthy** and **Synced**, meaning the live cluster state matches Git exactly. Whenever Jenkins pushes a new image tag, Argo CD detects the difference and rolls out the change automatically, which gives an audit trail (Git history), easy rollback (revert a commit) and protection against manual drift.

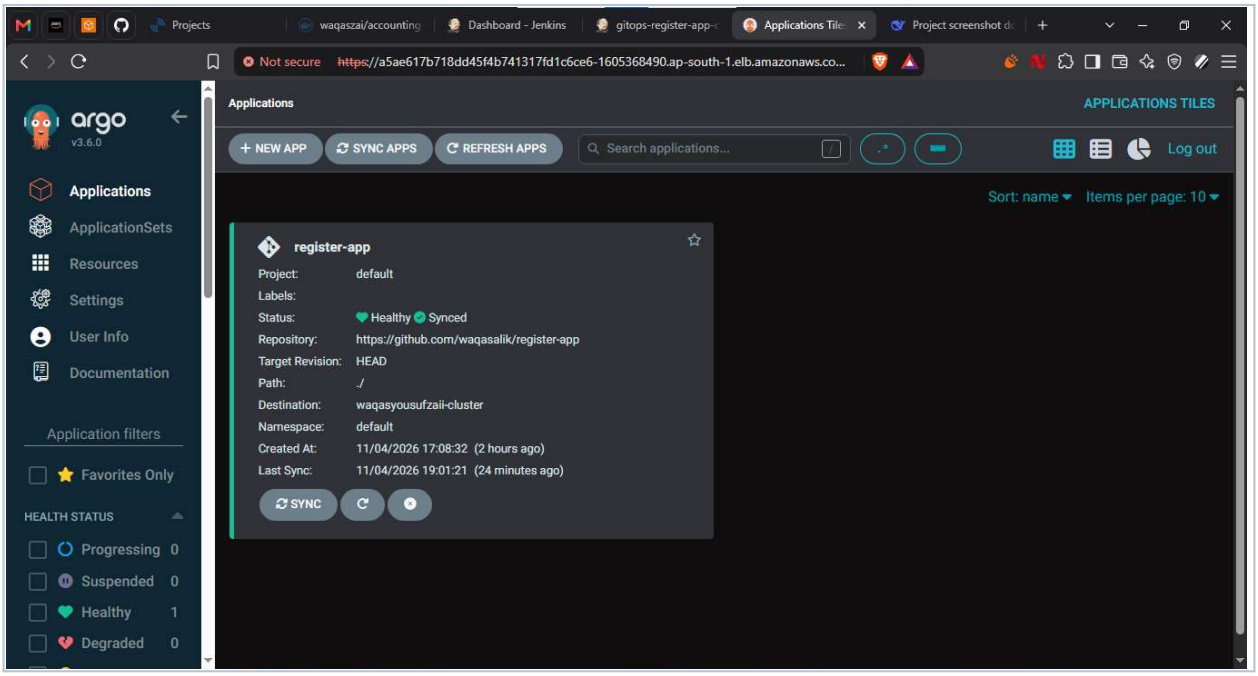


Figure 18: Argo CD dashboard: application register-app is Healthy and Synced.

9. Deployed Application

The final proof of success is the running application. It is opened through the DNS name of the AWS Elastic Load Balancer, which forwards traffic to the pods on EKS. The Mini Accounting App welcome page confirms the deployment and provides three buttons to test the REST API: **Check Health**, **View Summary** and **View Transactions**.

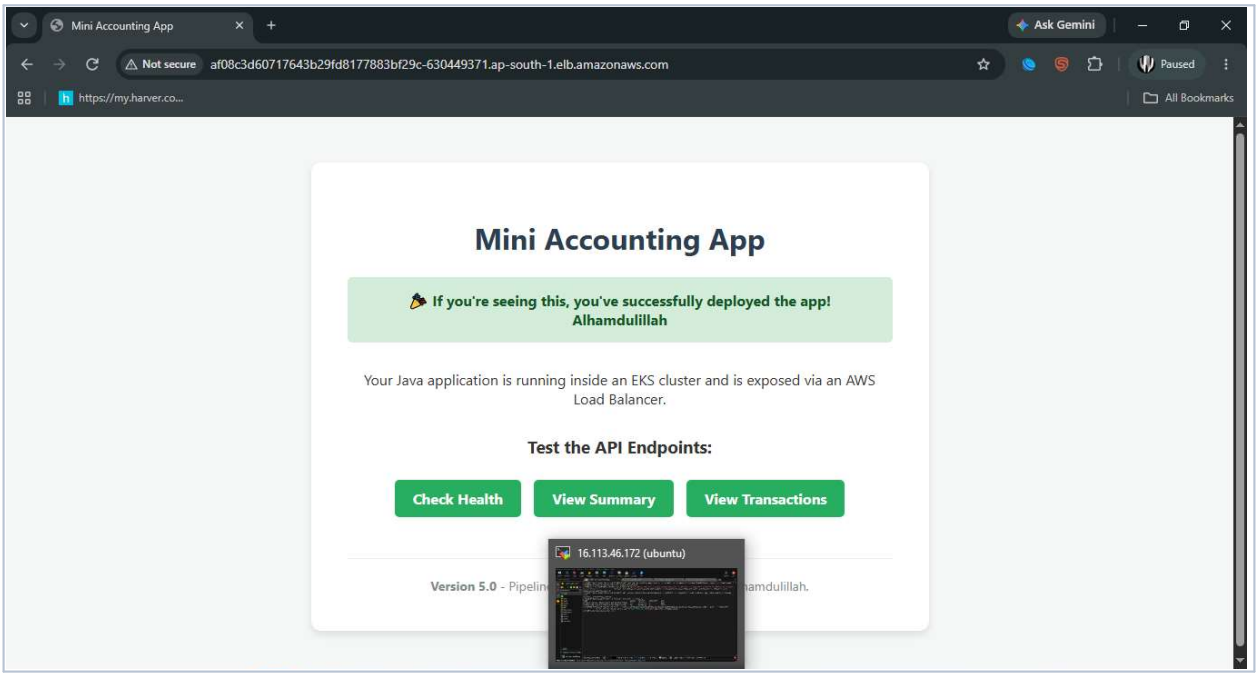


Figure 19: Mini Accounting App running on EKS and served through an AWS Load Balancer.

10. End-to-End Flow Summary

Step	Event	Tool	Result
1	Developer pushes code to main	GitHub	Webhook notifies Jenkins
2	Build and unit tests	Jenkins + Maven	Application packaged
3	Static analysis and Quality Gate	SonarQube	Gate Passed; ratings A/A/A
4	Docker image built and pushed	Docker, Docker Hub	waqaszai/accounting-app:1.0.0-26 and latest
5	Vulnerability scan	Trivy	Report of LOW/MEDIUM OS-level findings
6	CI triggers the CD job with IMAGE_TAG	Jenkins	gitops-register-app-cd starts
7	Manifest updated and committed	Jenkins + Git	deployment.yaml points to the new tag
8	Change detected and applied	Argo CD	Status Healthy and Synced
9	Pods rolled out, traffic exposed	Amazon EKS + ELB	Application reachable on the public URL

11. Observations and Recommendations

The pipeline works end to end. The following improvements would bring it closer to production standards:

- **Add unit tests and coverage reporting.** SonarQube shows 0.0% coverage on new code against a typical 80% target; add JaCoCo so the Quality Gate can enforce it.
- **Secure the tools with HTTPS.** Jenkins, SonarQube and Argo CD are accessed over plain HTTP (browser shows 'Not secure'). Put them behind TLS certificates or a reverse proxy.
- **Make Trivy a gate.** Configure it to fail the build on HIGH or CRITICAL findings and use a slimmer, regularly updated base image to reduce OS-level vulnerabilities.
- **Restrict network access.** Limit security-group rules for Jenkins (8080), SonarQube (9000) and Argo CD to trusted IP ranges.
- **Keep credentials in Jenkins Credentials only.** The pipeline already masks the Git and Docker secrets; continue to avoid placing tokens in scripts.
- **Avoid duplicate image builds.** The CD job rebuilds and rescans an image; promoting the exact image built in CI would guarantee that what was tested is what is deployed.
- **Naming consistency.** Rename jobs such as 'resgister-app-ci' and align Java versions across the README and Jenkins tool configuration.

12. Conclusion

This project demonstrates a modern, fully automated DevOps workflow. A single **git push** results in a tested, quality-checked and security-scanned container image, a Git-tracked manifest update and an automatic, self-healing deployment on Kubernetes. By combining Jenkins for CI, SonarQube and Trivy for quality and security, Docker Hub for artifact storage, and Argo CD with Amazon EKS for GitOps-based delivery, the solution is repeatable, auditable and easy to extend.